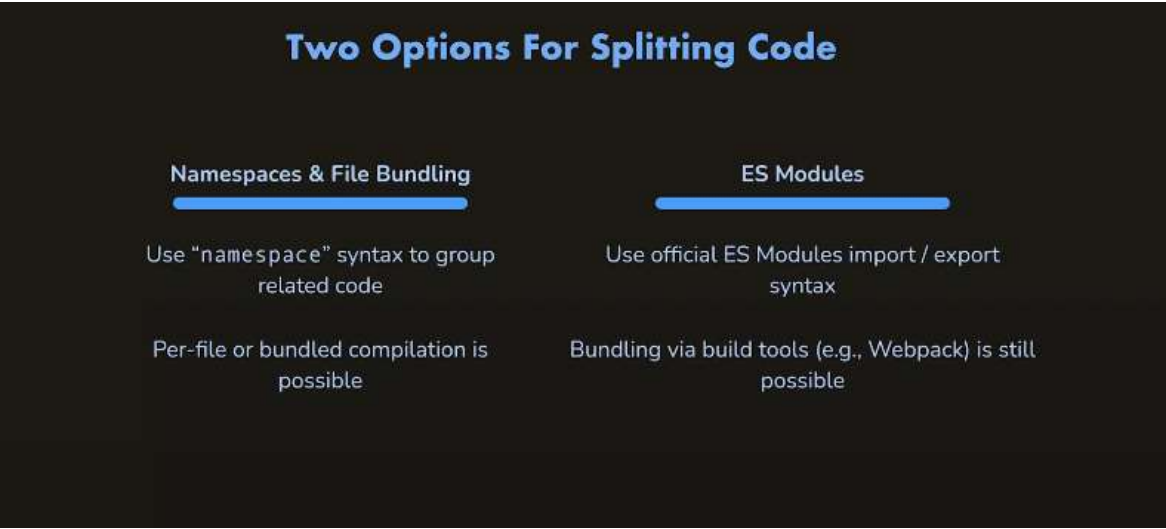




Splitting TypeScript Code Across Multiple Files

When working with larger TypeScript projects, you'll need to split your code across multiple files and then connect them together. TypeScript gives you **two main approaches** for this:

- 1. **Namespaces** (TypeScript-specific)
- 2. **ES Modules** (Standard JavaScript approach)

1. Namespaces + File Bundling (TypeScript Approach)

TypeScript provides a feature called **namespaces** to group related code together, even when that code lives in different files.

- You use the namespace keyword to logically group code.
- You can reference code from other files using the `/// <reference />` directive or by organizing files properly.
- With the help of the `tsconfig.json` file, TypeScript can **bundle** all these separate files into a **single JavaScript output file** during compilation.

This approach was especially useful before modern JavaScript module systems became widespread. It allows you to split code during development while ending up with one consolidated file for production.

2. ES Modules (Modern Standard Approach)

TypeScript also fully supports the official **ES Modules** syntax (`import / export`), which is the current standard way of splitting and sharing code in JavaScript.

- This is **not** TypeScript-specific — it's the native JavaScript module system.
- You use `export` to make features available from a file and `import` to use them in another file.
- Modern bundlers (like Webpack, Vite, esbuild, etc.) are typically used to bundle these modules for production.

Summary

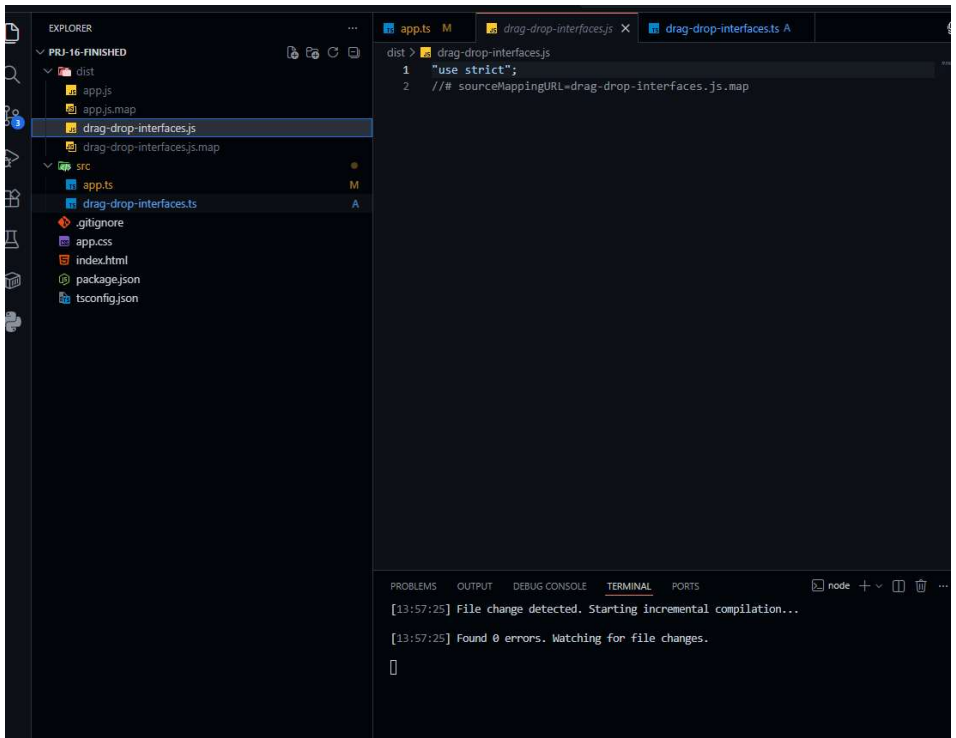
Approach	TypeScript Specific	Modern Standard	Output Strategy	Best For
Namespaces	Yes	No	Built-in bundling (tsconfig)	Legacy projects, simple apps
ES Modules	No (but supported)	Yes	Usually requires a bundler	Modern web/apps, scalability

Key Takeaway: You can split your TypeScript code across many files for better organization and maintainability. Then, depending on your needs and project age, you can either use **namespaces** (with TypeScript's built-in bundler) or the modern **ES Modules** syntax (with tools like Vite/Webpack).

Working with Namespace

The screenshot shows a code editor with a file named `drag-drop-interfaces.ts`. The code defines a namespace `App` and exports two interfaces: `Draggable` and `DragTarget`.

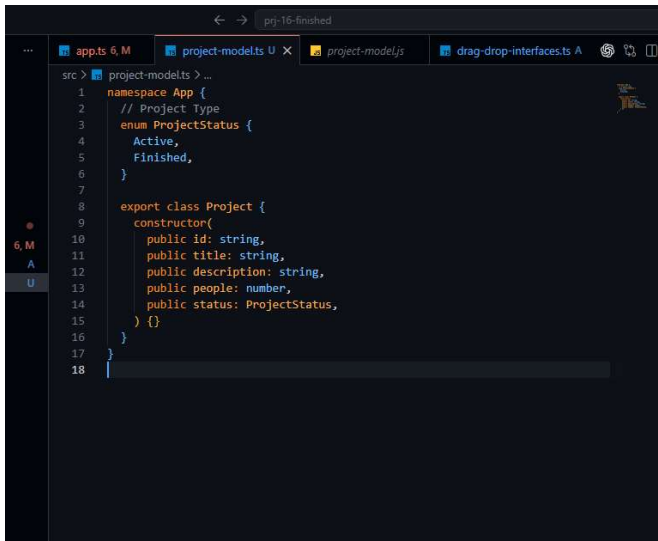
```
src > drag-drop-interfaces.ts > {} App
1 // Drag & Drop Interfaces
2 namespace App {
3   export interface Draggable {
4     dragStartHandler(event: DragEvent): void;
5     dragEndHandler(event: DragEvent): void;
6   }
7
8   export interface DragTarget {
9     dragOverHandler(event: DragEvent): void;
10    dropHandler(event: DragEvent): void;
11    dragLeaveHandler(event: DragEvent): void;
12  }
13 }
14
```



There is no js equivalent for interfaces. That's why this compiled file is empty.

But what if we create another namespace in a separate file and that contains ts code which has js equivalent ?

For example, in the project-model.ts file, we have defined enum and class,



Now, in the compiled code, we can see it's corresponding js code. But the question is will it work here?

```
dist > project-models > ...
1  "use strict";
2  var ProjectStatus;
3  (function (ProjectStatus) {
4    ProjectStatus[ProjectStatus["Active"] = 0] = "Active";
5    ProjectStatus[ProjectStatus["Finished"] = 1] = "Finished";
6  })(ProjectStatus || (ProjectStatus = {}));
7  class Project {
8    constructor(id, title, description, people, status) {
9      this.id = id;
10     this.title = title;
11     this.description = description;
12     this.people = people;
13     this.status = status;
14   }
15 }
16 ///  
sourceMappingURL=project-model.js.map
```

184. Working with Namespaces

Title

fadsf

Description

afdadfsf

People

5

ADD PROJECT

<

ACTIVE PROJECTS

FINISHED PROJECTS

Elements

Console

1

top

1 hidden

Uncaught TypeError: Cannot read property 'Active' of undefined

at ProjectState.addProject (app.ts:41)

at ProjectInput.submitHandler (app.ts:349)

We can see an error, even though TypeScript is not complaining.

```
41:48 PM] File change detected. Starting incremental compilation...
So the problem we have here is that in TypeScript world,
```

Everything is great

```
43 PM Found 0 errors. Watching for file changes.
because TypeScript is able to find all the things we need.
```

```
/// <reference path="drag-drop-interfaces.ts" />
/// <reference path="project-model.ts" />

namespace App {

  // Project State Management
  type Listener<T> = (items: T[]) => void;

  class State<T> {
    protected listeners: Listener<T>[] = [];

    addListener(listenerFn: Listener<T>) {
      this.listeners.push(listenerFn);
    }
  }

  class ProjectState extends State<Project> {
```

2: node

4:48 PM] File change detected. Starting incremental compilation because we import that model.

change detected. Starting incremental compilation because we import that model, but import here really just means

File change detected. Starting incremental compilation because we tell TypeScript where to find that type

change detected. Starting incremental compilation because once it is compiled to JavaScript,

This connection is basically destroyed.

Found 0 errors. Watching for file changes. So in JavaScript code when it executes

cp /tmp/diabetes-slides.html /mnt/c/Users/BJIT/Documents/diabetes-slides.html && echo "Copied OK" && wc -l /tmp/diabetes-slides.html

PROBLEMS OUTPUT TERMINAL ... 2: node
and when we then try to create a new project

```
src > app.ts > {} App > ProjectState > addProject > newProject
28   }
29   this.instance = new ProjectState();
30   return this.instance;
31 }
32
33
34 addProject(title: string, description: string, numofPeople: number) {
35   const newProject = new Project(
36     Math.random().toString(),
37     title,
38     description,
39     numofPeople,
40     ProjectStatus.Active
41   );
42   this.projects.push(newProject);
43   this.updateListeners();
44 }
```

3:41:48 PM] File change detected. Starting incremental compilation...

JavaScript doesn't find this project class

or constructor function.

So we have to make sure we carry over disconnection.

And for that we can go to the TS (mumbles)

and there set this out file option.

and the idea behind the out file

is that you tell TypeScript

that it should concatenate namespaces.

```
src > app.ts > ...
1  /// <reference path="drag-drop-interfaces.ts" />
2  /// <reference path="project-model.ts" />
3
4  namespace App {
5
6      // Project State Management
7      type Listener<T> = (items: T[]) => void;
8
9      class State<T> {
10         protected listeners: Listener<T>[] = [];
11
12         addListener(listenerFn: Listener<T>) {
13             this.listeners.push(listenerFn);
14         }
15     }
16
17     class ProjectState extends State<Project> {
```

PROBLEMS OUTPUT TERMINAL ... 2: node

So these references,

[3:43:19 PM] File change detected. Starting incremental compilation...

which it has during compilation

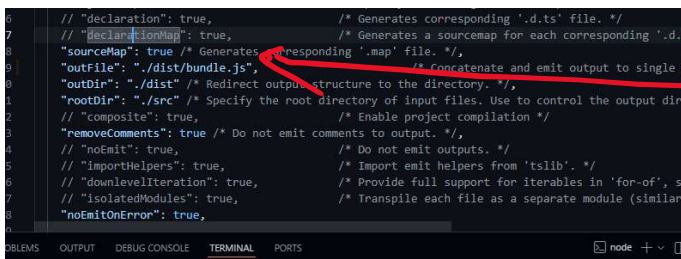
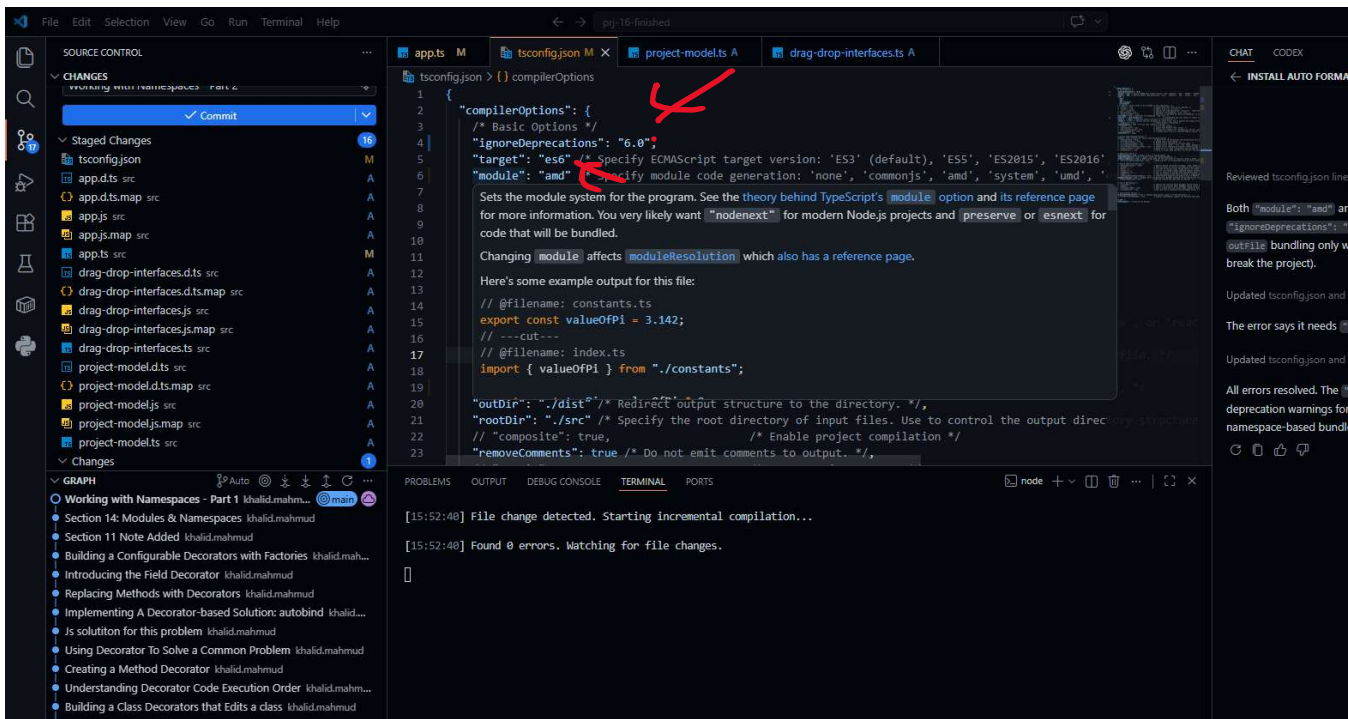
into one single JavaScript file

instead of compiling multiple JavaScript files.

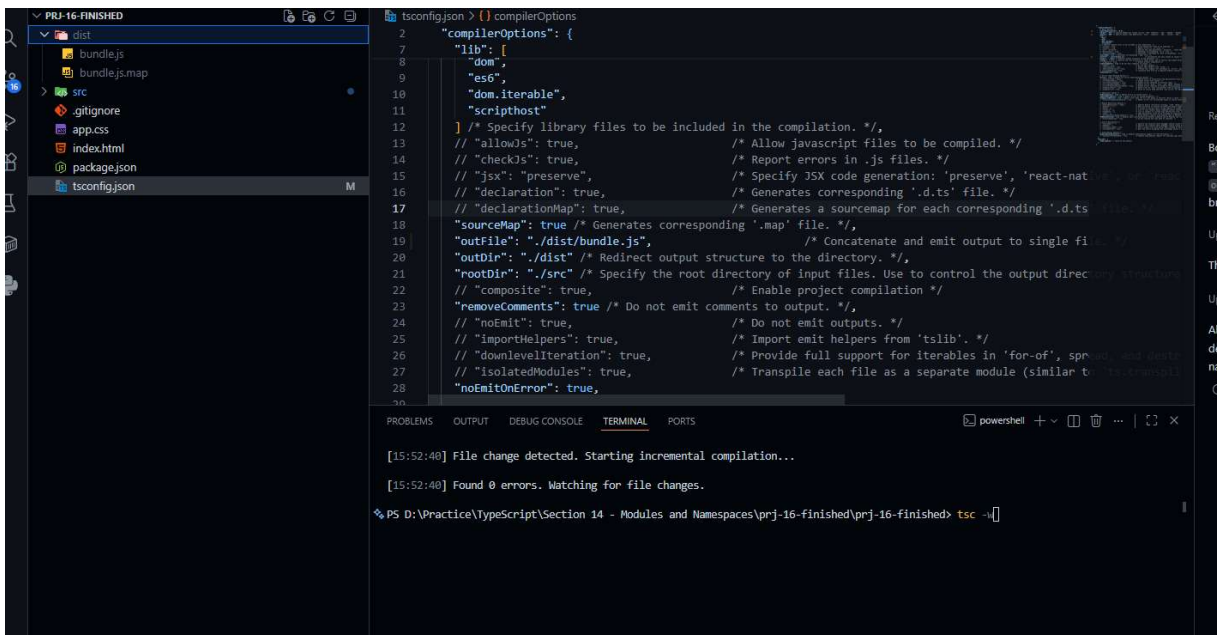
```
tsconfig.json > {} compilerOptions > abc outFile
13  // "checkJs": true, /* Report errors in .js files. */
14  // "jsx": "preserve", /* Specify JSX code generation:
15  // "declaration": true, /* Generates corresponding '.d.ts'
16  // "declarationMap": true, /* Generates a sourcemap for each
17  "sourceMap": true /* Generates corresponding '.map' file. */
18  "outFile": "./dist/bundle.js", /* Concatenate and emit
19  "outDir": "./dist" /* Redirect output structure to the directory. */,
20  "rootDir": "./src" /* Specify the root directory of input files. Use to co
21  // "composite": true, /* Enable project compilation */
22  "removeComments": true /* Do not emit comments to output. */,
23  // "noEmit": true, /* Do not emit outputs. */
24  // "importHelpers": true, /* Import emit helpers from 'tsl
25  // "downlevelIteration": true, /* Provide full support for iter
26  // "isolatedModules": true, /* Transpile each file as a sepa
27  "noEmitOnError": true,
28
29  /* Strict Type-Checking Options */
30  "strict": true /* Enable all strict type-checking options. */
```

PROBLEMS OUTPUT TERMINAL ... 2: node

which we named maybe bundle.js



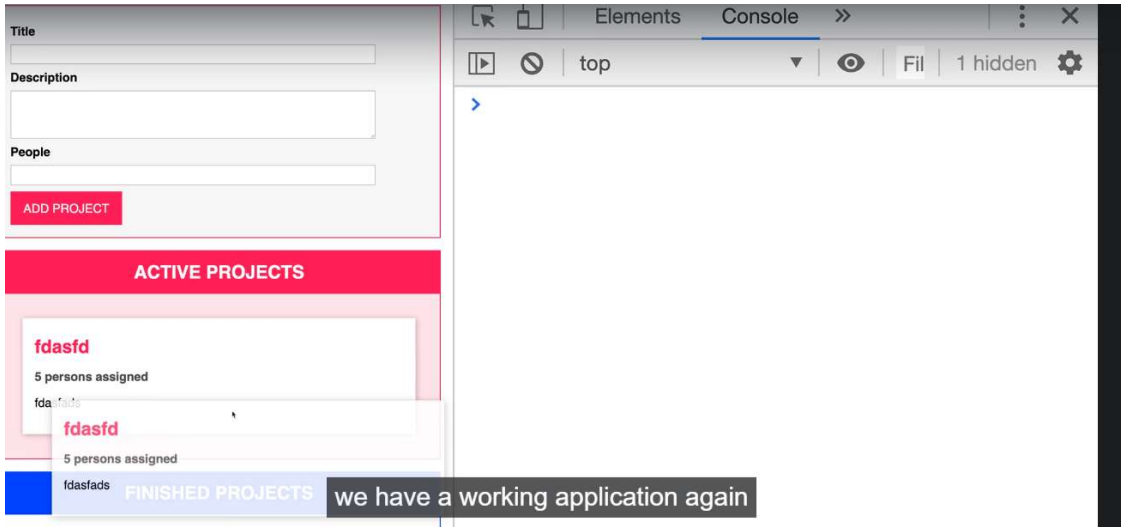
Now if we re-start our server, only bundle.js file is created



```
index.html / html / head / script
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scal
6     <meta http-equiv="X-UA-Compatible" content="ie=edge" />
7     <title>ProjectManager</title>
8     <link rel="stylesheet" href="app.css" />
9     <script src="dist/bundle.js" defer></script>
10  </head>
11  <body>
12    <template id="project-input">
13      <form>
14        <div class="form-control">
15          <label for="title">Title</label>
16          <input type="text" id="title" />
17        </div>
18        <div class="form-control">
```

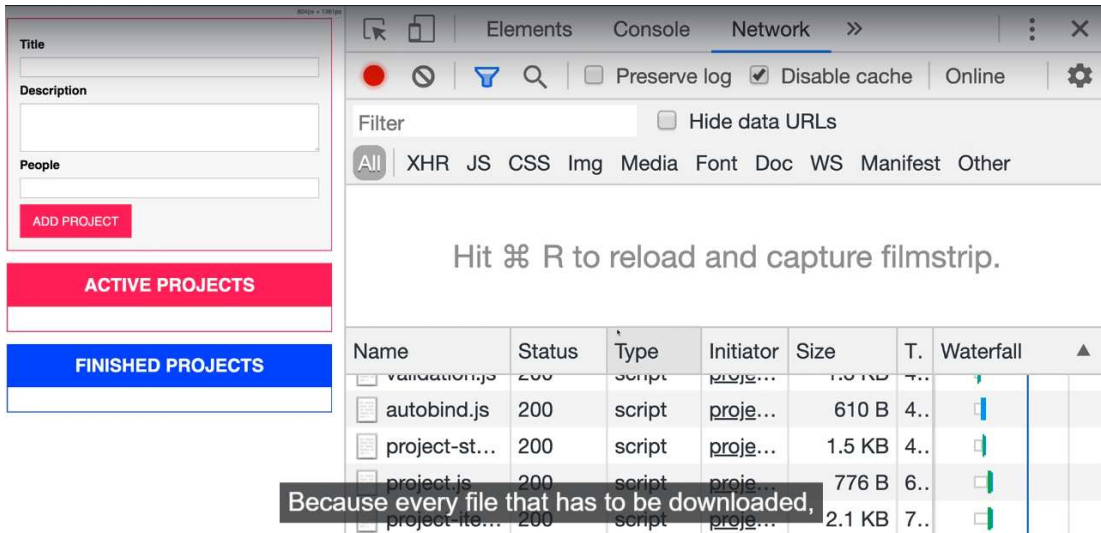
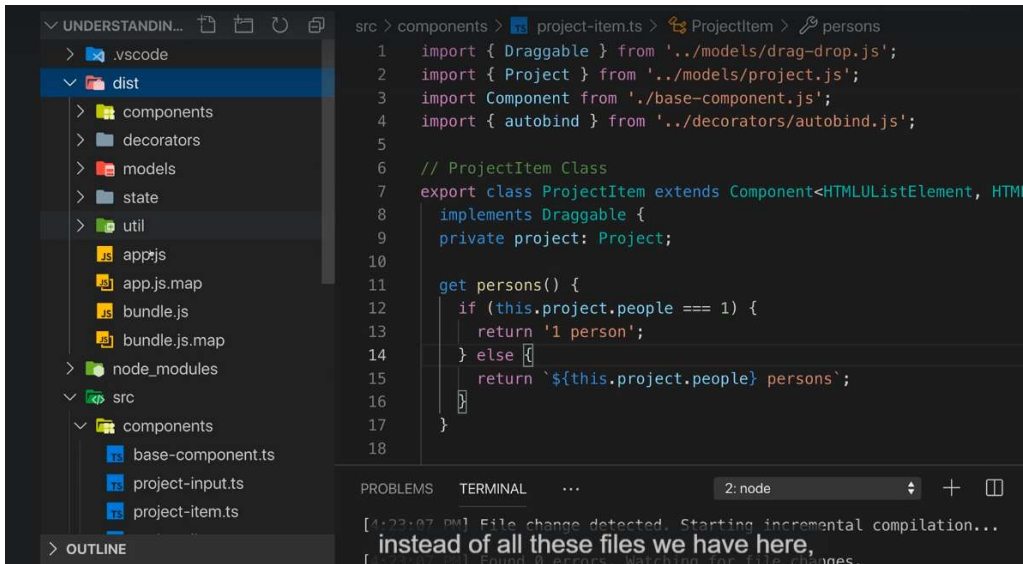
PROBLEMS 2 OUTPUT TERMINAL ... 2: node

[3:45:01 PM] Starting compilation in watch mode...
we just have to import bundle JS here instead of App JS.



Wrap Up


```
src > components > ts project-item.ts > ...
1 import { Draggable } from '../models/drag-drop.js';
2 import { Project } from '../models/project.js';
3 import Component from './base-component.js';
4 import { autobind } from '../decorators/autobind.js';
5
6 // ProjectItem Class
7 export class ProjectItem extends Component<HTMLUListElement, HTMLUListElement> {
8   implements Draggable {
9     private project: Project;
10
11     get persons() {
12       if (this.project.people === 1) {
13         return '1 person';
14       } else {
15         return `${this.project.people} persons`;
16       }
17     }
18   }
19 }
```



Why Modules Matter in TypeScript

And that's it for modules!

Using modules is an extremely useful and important feature in TypeScript. It allows you to write **more maintainable and manageable code**. Putting everything in a single file quickly becomes a mess and is only realistic for very tiny projects.

Whether you choose **namespaces** or **ES modules**, both approaches let you:

- Keep individual files relatively small and focused.
- Still build one large, cohesive application in the end.

Recommendation: Use ES Modules

When comparing the two approaches, the **clear recommendation** is to use **ES Modules**.

Why ES Modules are better:

- You get stronger type safety.
- Every file must **explicitly** declare what it imports and exports.
- Dependencies are clear and visible.

Problems with Namespaces:

- One file can import something that another file silently depends on.
- If you remove the file that does the importing, other files can break without any compile-time warning.
- These issues often only surface at runtime.

When to still use Namespaces?

- In very small projects.
- In legacy setups where you cannot use ES modules or a bundler.

Browser Compatibility & Bundling

The ES Modules syntax we're using right now only works in **modern browsers**. Older browsers like Internet Explorer 9 do not support it (and may not support other modern JavaScript features either).

Solution: Use a bundler

To make your TypeScript application work across all browsers, you should use a bundling tool such as **Webpack** (or modern alternatives like Vite, esbuild, etc.).

Benefits of bundling:

1. **Browser Compatibility** — The bundler converts modern module syntax into code that older browsers can understand.
2. **Performance** — Instead of downloading many small files, the browser only needs to fetch **one** JavaScript file.
 - Every file request has overhead (connection setup, HTTP request latency).
 - You can see this clearly in the browser's **Network tab** as a waterfall chart — each request has extra "white space" (setup time) at the beginning.

Even if browser support wasn't an issue, reducing the number of requests is generally better for performance.

Summary

- **Development:** Split your code into many small, well-organized TypeScript files using ES Modules.
- **Production:** Bundle everything into one (or very few) JavaScript files using Webpack or similar tools.

This gives you the best of both worlds: excellent developer experience with strong typing + optimal performance and compatibility for users.

Would you like me to format the next section, or combine everything we have so far into one complete, well-structured guide